

PRODUCT SPECIFICATION

Life Organization System

A personal operating system that turns what you save, learn, and build into completed projects, connected knowledge, and content worth sharing.

Version 1.0 | July 2026

Product thesis

The product is not another generic notes app. It is an execution and meaning-making system that connects discovery, learning, building, documentation, publishing, and personal identity.

Prepared from the Life Organization System concept brief and expanded into an implementation-ready product and design specification.

Executive Summary

The Life Organization System is a personal operating system for managing the different aspects of one person's life without fragmenting them into disconnected tools. It is designed around four reinforcing identities: personal brand, content creator, software developer, and student. The application converts everyday inputs - ideas, links, videos, tools, lessons, and project work - into structured projects, durable knowledge, and publishable outputs.

The differentiating loop is:

Capture -> Organize -> Act -> Document -> Learn -> Publish -> Reflect

The product succeeds when the user does not merely collect information, but repeatedly turns information into action and evidence. A saved AI tool should become a tested tool. A test should become a project log. A project log should become a knowledge note or content seed. A completed project should strengthen the user's portfolio and clarify their personal brand.

North-star outcome

The user can look at the system and clearly answer: What matters now? What should I do next? What am I learning? What have I finished? What can I share? What story is my work creating?

Document Map

Section	Purpose
1. Product foundation	Vision, user goals, principles, positioning, and success criteria.
2. Information architecture	Core entities, relationships, hierarchy, and lifecycle states.
3. Functional specification	Required product capabilities and acceptance criteria.
4. Interaction design	Navigation, capture, AI review, search, and weekly review flows.
5. Screen and visual design	Desktop/mobile layouts, components, and design system.
6. Technical architecture	Database, document storage, AI pipeline, search, and security.
7. Delivery plan	MVP, phased roadmap, analytics, risks, and open decisions.

1. Product Foundation

1.1 Problem Statement

A person who is simultaneously learning, building, publishing, applying for opportunities, and developing a personal identity often spreads their life across notes apps, bookmarks, project trackers, browser tabs, cloud folders, and social-media drafts. Each tool stores an isolated fragment. The user spends energy remembering where things live, repeatedly re-organizing, and rediscovering material they already saved.

The deeper problem is not storage. It is transformation. Most systems help collect information, but they do not reliably turn:

- saved material into tested knowledge
- ideas into completed projects
- project work into reusable documentation
- learning into applied skill
- activity into content and portfolio evidence
- repeated themes into a coherent personal identity

1.2 Product Vision

Vision statement

Create a calm, intelligent personal workspace where every discovery, lesson, and project can contribute to execution, understanding, creative output, and a cohesive life narrative.

1.3 Target User

The initial user is an ambitious individual managing several overlapping roles. The design is intentionally optimized for a single-user system before expanding to collaboration.

Attribute	Description
Primary profile	Student, early-career builder, developer, researcher, creator, or multidisciplinary knowledge worker.
Behavior	Saves many tools and resources, starts projects, learns continuously, and wants to publish or document work.
Pain	Information is scattered; saved items are rarely revisited; project documentation is inconsistent; goals feel disconnected.
Desired result	A reliable system that reduces mental overhead and makes progress, learning, and identity visible.

1.4 User Goals and Product Responses

User goal	Product response	Why it helps
Develop a personal brand	Identity Studio, recurring-theme analysis, evidence-linked profile, and published-output history.	The brand emerges from actual work and repeated interests instead of invented adjectives.
Create quality content	Content seeds generated from project logs, tested tools, knowledge notes, and retrospectives.	Content creation becomes a byproduct of meaningful activity, reducing blank-page pressure.
Become a stronger developer	Project pages, engineer logs, experiments, decisions, repositories, retrospectives, and applied knowledge.	The system captures proof of technical reasoning and completed work.
Succeed as a student	Knowledge notes, mastery states, source links, prerequisite relationships, and applied examples.	Learning becomes durable and connected to real use rather than passive note storage.
Maintain social and personal balance	Life Areas and weekly review make	The system supports balance while

User goal	Product response	Why it helps
	neglected responsibilities visible without turning the app into a rigid schedule.	preserving user autonomy.

1.5 Product Principles

Principle	Design implication
Transformation over collection	Every major object should support a next step. Saved tools can be tested; notes can be applied; logs can become content.
Structured objects, flexible documents	Metadata and relationships live in typed records; expressive writing lives in blocks or Markdown-compatible documents.
AI proposes, the user approves	AI can classify and generate, but canonical data changes are previewed to preserve trust and quality.
One activity can serve multiple identities	Projects and notes may connect to several Life Areas and goals instead of being forced into one folder.
Action before aesthetics	Home and project screens prioritize next actions, stale work, and review queues over decorative dashboards.
Portable by design	The user can export human-readable Markdown and assets, reducing lock-in.
Calm, not empty	The interface is visually restrained but information-rich, with clear hierarchy and keyboard-friendly actions.

1.6 Product Positioning

Recommended positioning statement:

Positioning

A personal operating system that turns what you save, learn, and build into completed projects, connected knowledge, and content worth sharing.

The product is differentiated from conventional tools by combining an opinionated transformation workflow with portable, flexible documents. It should not compete by recreating every feature of Notion, Obsidian, a task manager, a read-it-later app, and a social scheduler. It should connect the useful parts of those categories around a single personal growth loop.

1.7 Success Metrics

Metric	Definition	Why it matters
Weekly active transformation rate	Percentage of weeks in which at least one capture becomes a project, note, tested tool, or content seed.	Measures the core promise rather than raw capture volume.
Active-project next-action coverage	Percentage of active projects with a defined next action.	Indicates whether projects are executable.
Capture review latency	Median time from capture to approve, dismiss, or organize.	Shows whether the Inbox remains trustworthy.
Tool test conversion	Percentage of saved tools that reach Tested, Adopted, or Rejected.	Prevents the Library from becoming a link graveyard.
Knowledge application rate	Percentage of knowledge notes linked to at least one project, experiment, or content item.	Measures applied learning.
Project completion rate	Percentage of started projects completed within the chosen review period.	Measures follow-through.
Output generation rate	Published or portfolio-ready outputs per completed project.	Measures whether work strengthens the user's public narrative.

2. Information Architecture

2.1 Core Object Model

```

Life Area
|-- Goals
|-- Projects
|   |-- Tasks and milestones
|   |-- Engineer log entries
|   |-- Knowledge notes
|   |-- Tools and sources
|   |-- Content outputs
|
|-- Knowledge notes

Capture
|-- Source
|-- Tool
|-- Project idea
|-- Knowledge note
|-- Content idea

```

The architecture separates object identity from document content. A Project is not merely a page; it has a lifecycle, relations, next action, dates, and outputs. Its description and logs can still use an expressive block editor.

2.2 Life Areas

Life Areas represent long-term identities or responsibilities with no natural completion date. Initial defaults should include Personal Brand, Content, Software Development, Student, Social, and Personal. Users may rename or add Areas.

Field	Type	Requirement
id	UUID	Required, immutable.
name	string	Required, unique per user.
description	text	Optional identity or responsibility statement.
icon	string	Optional restrained icon.
accent	token	Optional design-system accent.
status	active inactive	Required.
created_at / updated_at	timestamp	Required.

2.3 Goals

Goals create measurable direction inside Areas. They answer why a project matters. A project may support multiple goals, while each goal has one primary Area.

Field	Type	Notes
title	string	Outcome or identity statement.
type	outcome habit identity	Determines review and progress behavior.
timeframe	quarter year long-term	Optional planning horizon.
metric_name	string	Optional metric label.
target_value/current_value	number	Optional quantitative progress.
status	active paused completed	Required.
area_id	UUID	Primary Area relation.

2.4 Projects

Projects are finite efforts intended to create an outcome. Every Active project must have a next action. Project status is deliberately explicit: Idea, Planned, Active, Paused, Completed, or Archived.

Field	Type	Behavior
title	string	Required.
summary	text	One-sentence outcome.
status	enum	Controls default list grouping and review prompts.
area_ids	many-to-many	Allows one project to advance several identities.
goal_ids	many-to-many	Explains why the project matters.
priority	low medium high	Optional; avoid false precision.
start_date / target_date	date	Optional planning metadata.
next_action	string	Required when Active.
repository_url / live_url	URL	Optional output links.
cover_asset_id	UUID	Optional; secondary to actionable metadata.
completed_at	timestamp	Set when completed.
body_document_id	UUID	Flexible overview document.

2.5 Project Log Entries

Engineer logs are separate records so they can be filtered, searched, linked, and converted into outputs. Entry types are Progress, Decision, Experiment, Problem, Learning, and Reflection.

Field	Type	Behavior
project_id	UUID	Required parent project.
entry_type	enum	Provides templates and later filtering.
title	string	Optional; generated from first line when absent.
body_document_id	UUID	Block document with full entry.
created_at / updated_at	timestamp	Preserves chronology.
content_seed_status	none suggested created	Supports the work-to-content pipeline.

2.6 Knowledge Notes

Knowledge Notes contain durable explanations and mental models. They should separate AI-generated summaries from the user's own understanding. Mastery states are Captured, Learning, Understood, and Applied.

Field	Type	Behavior
title	string	Required concept name.
note_type	concept how-to reference comparison question mental-model	Selects a useful template.
mastery	captured learning understood applied	Makes learning progress visible.
body_document_id	UUID	Contains explanation, examples, mistakes, and user wording.
area_ids	many-to-many	Supports topic browsing.
project_ids	many-to-many	Shows practical application.
source_ids	many-to-many	Maintains provenance.
ai_sections	structured metadata	Labels generated summaries separately from user-authored sections.

2.7 Captures and Inbox

A Capture is immutable raw input added before its destination is known. The original input remains accessible even after AI processing. One capture can produce multiple proposed objects.

Field	Type	Behavior
raw_input	text	Original text or extracted payload.
input_type	text URL image audio file	Determines ingestion pipeline.
source_url / platform	string	Optional source metadata.
processing_status	unprocessed processing ready failed	Supports asynchronous ingestion and retry.
classification	project idea tool knowledge content idea task reference other	AI suggestion, not final truth.
review_status	pending accepted partially accepted dismissed	Controls Inbox queue.
proposal_bundle_id	UUID	Links generated proposed changes.

2.8 Tools and Test Later

Tools move through a deliberate test lifecycle: Saved, Shortlisted, Testing, Tested, Adopted, or Rejected. Each saved tool should include an expected use so that collection remains purposeful.

Field	Type	Behavior
name / URL	string	Required name; URL optional.
status	enum	Controls the Test Later workflow.
expected_use	text	Prompted at save or shortlist.
test_criteria	text	Defined when Testing begins.
verdict	text	Required before Tested, Adopted, or Rejected.
rating	1-5	Optional; subordinate to written verdict.
project_ids	many-to-many	Connects tools to real use.
source_id	UUID	Tracks where it was discovered.
tested_at	timestamp	Supports review and content prompts.

2.9 Sources

Sources represent external material consumed by the user. A Source is distinct from the knowledge created from it. Supported types include YouTube, TikTok, Instagram, article, paper, book, podcast, and other.

Field	Type	Behavior
title / URL	string	Canonical source identity.
platform	enum	Chooses fetch or transcription pipeline.
author / published_at	metadata	Optional provenance.
transcript	text/object	Stored when legally and technically available.
summary	text	AI-generated and labeled.
capture_id	UUID	Links back to original ingestion.
processing_version	string	Allows reprocessing after prompt/model changes.

2.10 Content Items

Content Items distinguish an idea from an actual publication. Statuses are Idea, Draft, Review, Scheduled, Published, and Archived. Content must preserve provenance back to projects, notes, tools, logs, and sources.

Field	Type	Behavior
title	string	Required working title.

Field	Type	Behavior
status	enum	Pipeline stage.
format	short post thread video article newsletter case study	Chooses drafting template.
channel	LinkedIn X Medium Substack YouTube Instagram TikTok Portfolio	Optional destination.
body_document_id	UUID	Draft content.
origin relations	many-to-many	Shows what work created the content.
published_url / published_at	metadata	Evidence of output.

2.11 Relationship Model

Use dedicated join tables for high-volume, strongly typed relationships such as Project-Area and Project-Goal. Use a generic relation table for semantic links such as learned-from, used-in, prerequisite-of, inspired-by, and created-from.

```
relation {
  source_type, source_id,
  target_type, target_id,
  relation_type,
  created_at
}
```

Rule

Everything can connect, but not everything is the same thing. Typed entities preserve useful workflows; flexible relations preserve discovery.

2.12 Document and Block Model

Documents provide the flexible writing experience. Each document is composed of ordered blocks. Generic blocks include paragraph, headings, lists, checklist, code, quote, callout, image, file, embed, and table. Product-specific blocks include Decision, Experiment, Learning, Content Seed, Relation, Tool Card, and Project Log reference.

Product block	Required fields	Product value
Decision	Decision, rationale, alternatives	Creates interview-ready evidence of technical reasoning.
Experiment	Hypothesis, method, result, next test	Makes project learning reusable and comparable.
Learning	Lesson, own explanation, application	Moves notes toward mastery.
Content Seed	Hook, claim, audience, format	Turns work into a low-friction draft starting point.
Relation card	Object reference and relation type	Makes connected context visible inside documents.

3. Functional Specification

3.1 Universal Capture

A capture control is available globally through a keyboard shortcut, top-bar input, and mobile center action. The user can enter plain text, paste a URL, attach a file or image, record audio, or use command-like language.

Requirement	Acceptance criteria
Fast entry	Text or URL can be saved in no more than two explicit user actions.
Immutable original	The exact original input remains available after processing.
Automatic type detection	The system detects URL, media, file, and plain-text inputs.
Immediate confirmation	The app shows that the capture is saved before long processing completes.
Retry support	Failed processing can be retried without creating duplicate captures.
Contextual capture	When launched from a project, the user may save directly as a project log while retaining capture provenance.

3.2 AI Classification and Proposal Review

AI transforms a raw capture into a proposal bundle. A proposal bundle may include new entities, summaries, extracted concepts, suggested relations, and potential duplicate matches. The system must never silently create large sets of canonical records.

Requirement	Acceptance criteria
Preview	The user sees proposed objects and relationships before approval.
Granular approval	The user can accept all, accept selected, edit, or dismiss.
Duplicate detection	Potential matches are displayed with merge or create-new choices.
Source labeling	Generated summaries and extracted claims are visibly marked as AI-generated.
Confidence behavior	Low-confidence classifications are routed to review rather than auto-applied.
Audit trail	The system records which proposal created or modified each object.

3.3 Project Management

Capability	Specification
Project creation	Create manually, from a capture, or from "I have a new project idea..." language.
Status workflow	Idea → Planned → Active → Paused → Completed → Archived. Backward movement is allowed.
Activation rule	An Active project requires a next action; the UI prompts before activation.
Project page	Overview, Log, Tasks, Knowledge, Resources, Outputs, and Retrospective tabs.
Stale detection	Flag Active projects without a log update or task completion within a configurable period.
Completion flow	Prompt for retrospective, outputs, knowledge promotion, and portfolio/content opportunities.
Views	Board, list, timeline, by Area, recently updated, and portfolio-ready.

3.4 Engineer Log

Capability	Specification
Quick entry	Add a log from the project header or global capture.
Templates	Progress, Decision, Experiment, Problem, Learning, Reflection.
Chronology	Entries default to reverse chronological order with filters.
Relations	Entries can reference tasks, notes, tools, files, sources, and code links.
Content conversion	A log can generate one or more content seeds while preserving provenance.
Search	Logs are searchable by project, entry type, date, and semantic meaning.

3.5 Knowledge Management

Capability	Specification
Creation	Create manually, from a source, from project logs, or from AI extraction.
Learning template	Explanation, own words, example, common mistakes, application, prerequisites, and sources.
Mastery	Captured, Learning, Understood, Applied.
Contextual connections	Every note page shows used-in projects, sources, related notes, prerequisites, and outputs.
Backlinks	Show all objects that reference the note.
Graph	Secondary exploration mode; never required for basic navigation.
Promotion	Project learnings can be promoted into durable notes without copy-paste.

3.6 Tool Testing Workflow

Saved -> Shortlisted -> Testing -> Tested -> Adopted / Rejected

Transition	Required prompt
Save	Why might this be useful? Optional when using one-tap capture, required before Shortlisted.
Begin testing	What are you trying to accomplish? Which project could benefit? What defines success?
Finish testing	What worked? What failed? Would you use it again? What did you learn?
Adopt or reject	Record a concise verdict and evidence.
After test	Offer to create a knowledge note, project log, comparison, or content seed.

3.7 Content Pipeline

Seed -> Draft -> Review -> Ready / Scheduled -> Published

Capability	Specification
Create from work	Project log, test verdict, note, retrospective, or source can become a seed.
Suggested angles	AI proposes lesson, update, explainer, comparison, case study, or opinion formats.
Provenance	The content item retains links to all source work.
Channel adaptation	A canonical draft may create variants for selected channels.
Publishing	Initial MVP stores published URL and date; automated

Capability	Specification
	publishing is deferred.
Portfolio evidence	Projects can mark outputs as portfolio-ready and expose them in Identity Studio.

3.8 Weekly Review

The Review experience is a guided sequence rather than a passive dashboard. It should take the user through cleanup, decision-making, and focus selection.

1. Process captures awaiting review.
2. Review Active projects and identify stalled work.
3. Confirm or update the next action for every Active project.
4. Review tools recently tested or stuck in Testing.
5. Promote project learnings into durable knowledge.
6. Review content seeds created from the week's work.
7. Check progress against goals and neglected Life Areas.
8. Choose the next week's focus projects and intended outputs.

3.9 Search and Command Palette

Search combines exact text, metadata filters, relationship traversal, semantic retrieval, and natural-language interpretation. Example queries include "tools I saved but never tested," "things I learned from the Spotify project," and "active student projects with no next action."

Layer	Behavior
Exact search	Titles, body text, tags, URLs, and metadata.
Filter search	Type, Area, status, mastery, date, project, source, and channel.
Relationship search	Traverse created-from, used-in, learned-from, and supports relations.
Semantic search	Find conceptually similar notes, logs, and captures.
Command palette	Create, navigate, change status, add log, and invoke common workflows.

3.10 Identity Studio

Identity Studio is not a decorative profile page. It synthesizes evidence from the user's real work.

Module	Function
Identity statements	Capture what the user cares about, wants to be known for, and refuses to represent.
Emerging themes	Summarize recurring topics, project patterns, opinions, tools, and outputs over a selected period.
Evidence map	Connect claimed skills or themes to completed projects, logs, notes, and publications.
Portfolio queue	Surface completed work that lacks a case study, README, demo, or publishable explanation.
Brand consistency	Store tone, visual references, recurring phrases, and selected themes without forcing them into every screen.

4. Interaction Design

4.1 Primary Navigation

Home | Inbox | Projects | Knowledge | Library | Content | Review

Navigation is intentionally restrained. Life Areas are filters and context, not top-level destinations, because separating each identity into its own workspace would recreate the fragmentation the product is intended to solve.

4.2 Home Experience

Home answers three questions: What matters now? What should I do next? What requires review?

Section	Content	Primary action
Today	Pinned intentions, due items, and selected focus projects.	Open or complete next action.
Active project actions	One next action per Active project, ranked by focus and urgency.	Mark done or update action.
Inbox	Count and preview of captures awaiting approval.	Review proposals.
Test queue	Shortlisted or Testing tools that need attention.	Start or finish a test.
Recent learning	New notes and project learnings.	Add own explanation or application.
Content opportunities	Recent logs, tests, and completions with publishable potential.	Create seed.
Goal and Area balance	Progress plus neglected Areas, shown quietly.	Open weekly review.

4.3 Inbox Flow

[Original capture]

AI suggests:

- + Create Tool: Repomix
- + Link Project: AI Developer Workflow
- + Create Content Seed: Giving an agent codebase context

[Accept all] [Review individually] [Dismiss]

The original capture stays visually separate from proposals. The review interface supports inline editing before acceptance. Accepted changes show where each item was created.

4.4 Project Activation Flow

9. User changes a project from Planned or Idea to Active.
10. System checks for a clear outcome and next action.
11. If missing, an inline prompt requests the next physical action.
12. User may optionally choose target date and focus level.
13. Project appears in Home and weekly review.

4.5 Video Digestion Flow

Paste video URL

- > Save capture immediately
- > Fetch metadata / transcript

- > Generate source summary
- > Extract concepts
- > Match existing notes
- > Suggest new notes and relationships
- > User reviews
- > Approved knowledge enters graph

The application should not blindly create one note per extracted heading. It should prefer linking to existing concepts and present merge candidates.

4.6 Project Completion Flow

14. Confirm final outcome and links.
15. Complete a short retrospective.
16. Promote important learnings into Knowledge Notes.
17. Review reusable decisions and experiment evidence.
18. Create content or portfolio seeds.
19. Connect the completed project to demonstrated skills and goals.
20. Move unresolved tasks into another project or archive them explicitly.

4.7 Empty, Loading, Error, and AI States

State	Design requirement
Empty	Explain what belongs here, why it is useful, and provide one clear starter action.
Loading	Use skeleton structure matching the destination layout; show background processing status separately.
Error	State what failed, what remains safe, and whether the user can retry.
Partial AI result	Show available proposals while labeling failed extraction steps.
Offline	Allow local capture and queue processing for reconnection.
No search results	Offer filter removal, semantic search, or creation of a new item.

4.8 Notification Philosophy

Notifications should protect follow-through, not create guilt. Initial notifications may include: capture processing complete, project target approaching, tool test awaiting verdict, weekly review, and failed import. Streaks, constant reminders, and engagement-driven nudges are explicitly out of scope.

5. Screen and Visual Design

5.1 Visual Direction

The product should feel like a personal studio and operating environment rather than corporate project-management software. The design is calm, editorial, and precise.

Principle	Specification
Calm base	Warm off-white or near-white canvas, near-black text, soft neutral borders, and limited shadows.
One restrained accent	Accent is used for current focus, links, and selected Area context rather than coloring every object.
Document-first	Writing and thinking occupy the main canvas; metadata and relationships live in side panels.
Dense but breathable	Compact metadata and controls with generous space around primary reading and writing content.
Identity without clutter	Areas may have an icon and accent, but notes do not require decorative covers or emojis.
Action hierarchy	Next actions, review queues, and status changes are visually stronger than cover imagery.

5.2 Design Tokens

Token	Suggested value	Use
Canvas	Warm white #F8F7F3	Application background.
Surface	White #FFFFFF	Cards, documents, panels.
Primary text	#161616	Headings and body.
Secondary text	#66645F	Metadata and helper text.
Border	#DDDAD2	Dividers and low-emphasis structure.
Accent	#365E55	Links, focus state, selected controls.
Warning	#8A5A1F	Stalled work or review attention.
Error	#9A3838	Failures and destructive confirmations.
Radius	8 px cards; 6 px controls	Soft but not playful.
Spacing	4 px base scale	Consistent compact rhythm.

5.3 Typography

Role	Specification
Display	Modern grotesk or editorial sans, 28-36 px, medium weight.
Section heading	18-24 px, semibold.
Body	15-17 px, regular, comfortable line height.
Metadata	12-13 px, medium contrast.
Code	Monospace, 13-15 px.
Labels	11-12 px uppercase or sentence case; avoid excessive letter spacing.

5.4 Desktop Shell

Primary nav	Top bar: search / capture / context
Home Inbox Projects	Main content

Knowledge Library Content Review	Inspector
---	-----------

The Inspector is contextual and collapsible. It contains properties, connections, provenance, and history. It should not permanently reduce the main writing width on small desktop screens.

5.5 Project List Screen

Element	Design
Header	Title, current view, filter, sort, and Create Project.
Default grouping	Active, Planned, Ideas, Paused, Completed.
Project card	Title, Areas, status, next action, last log date, target, and output count.
Primary emphasis	Next action and staleness, not cover image.
Views	List default; board and timeline optional.
Empty state	Prompt the user to describe a project idea or import one from Inbox.

5.6 Project Detail Screen

Semantic Spotify Recommendations [Active v] Software Development · Content · Personal Brand	
NEXT ACTION Run a 25-song comparison against Spotify recommendations	
Overview Log Tasks Knowledge Resources Outputs	
Main document / selected tab	Project inspector
Objective	Target, goals, links
Current state	Related knowledge
Decisions / experiments	Tools and sources
	Output opportunities

Tabs prevent a mature project from becoming one unscannable page while preserving a central document experience. The next action remains visible near the top across tabs.

5.7 Knowledge Note Screen

Vector Embeddings AI/ML · Concept	Mastery: Applied
Plain-language explanation	Connections
How it works	Used in projects
Example	Learned from sources
Common mistakes	Related concepts
My explanation	Prerequisites
Where I applied it	Content outputs

"My explanation" and "Where I applied it" are first-class sections. Generated summaries are visually labeled and never substitute for demonstrated understanding.

5.8 Mobile Navigation and Capture

Home | Inbox | + | Projects | Library

The center action opens Write a thought, Paste a link, Scan an image, Record audio, Create project, and Add log entry. Mobile is optimized for rapid capture, reviewing AI proposals, seeing next actions, and recording test verdicts. Complex graph editing remains desktop-first.

5.9 Core Components

Component	Purpose
Object card	Consistent preview for project, note, tool, source, or content item.
Relation chip	Displays typed connection with hover or tap preview.
Status control	Compact state transition with required-field validation.
Next-action row	Prominent actionable line with complete and edit controls.
Proposal card	AI suggestion with provenance, confidence, edit, accept, and dismiss.
Document block menu	Insert generic and product-specific blocks.
Inspector panel	Properties, relations, provenance, versions, and export.
Review queue	Sequential decision interface with progress indicator.

5.10 Accessibility

- All status and relation meaning must be available without color.
- Keyboard navigation must cover global capture, search, project switching, and proposal approval.
- Focus states must be high contrast and persistent.
- Editor blocks require semantic heading structure and accessible drag alternatives.
- Graph view must have a list-based equivalent.
- AI confidence and generation labels must be expressed in text, not icon alone.
- Motion should respect reduced-motion preferences.

6. Technical Architecture

6.1 Recommended Stack

Layer	Recommendation	Rationale
Mobile client	Expo / React Native with Expo Router	Matches current development familiarity and supports iOS and Android.
Web client	Next.js or React web application	Best environment for the full block editor and dense desktop workspace.
Data fetching	TanStack Query	Caching, optimistic updates, retry, and server-state separation.
Local state	Zustand or equivalent	Limited UI state without over-centralizing server data.
Offline cache	SQLite on mobile	Reliable capture and basic reading while offline.
Backend	TypeScript API using Hono, Fastify, or Next.js routes	Shared types and straightforward deployment.
Database	PostgreSQL with Drizzle ORM	Typed relational data, flexible queries, transactions, and migrations.
Object storage	Cloudflare R2	Assets, imports, exports, and document snapshots.
Semantic search	pgvector	Avoids a separate vector database during early scale.
Jobs	Durable queue / worker system	Required for transcription, extraction, embeddings, and exports.

6.2 Storage Strategy

Markdown files in R2 should not be the sole source of truth. R2 is object storage and does not provide relational integrity, efficient joins, transactional updates, or robust concurrent editing. Use PostgreSQL as the canonical structured store and Markdown as a portable export and backup format.

```
PostgreSQL
  entities, relationships, statuses, metadata
  document block JSON or normalized blocks
  search indexes and embeddings
```

```
Cloudflare R2
  /assets/{asset_id}/{filename}
  /exports/projects/{project_id}.md
  /exports/notes/{note_id}.md
  /snapshots/{document_id}/{timestamp}.md
```

6.3 Markdown Export

```
---
id: project_spotify_recs
type: project
title: Semantic Spotify Recommendations
status: active
areas:
  - software-development
  - content
related_notes:
  - audio-embeddings
updated_at: 2026-07-22
---
```

```
# Semantic Spotify Recommendations
```

```
## Objective
```

```
Build a recommendation engine based on musical similarity...
```

Export identifiers must be stable. Relations should use stable IDs plus human-readable titles. Assets should export with relative links when producing a downloadable archive.

6.4 AI Processing Pipeline

```
Capture submitted
-> Save immutable raw capture
-> Detect input and source type
-> Fetch metadata / transcribe / parse
-> Produce structured extraction
-> Retrieve potential existing matches
-> Generate proposal bundle
-> User review
-> Transactional write of approved changes
-> Index text and relations
```

Pipeline requirement	Specification
Idempotency	Each processing step uses a stable job key to avoid duplicate outputs.
Versioning	Store prompt, model, parser, and extraction schema version.
Partial failure	Preserve successful stages and allow targeted retry.
Cost controls	Summarize and embed only when needed; cache source processing.
Safety	Validate generated structured output against schemas before proposal creation.
Observability	Record duration, failure reason, token usage, and user acceptance rate.

6.5 Search Architecture

- PostgreSQL indexes for IDs, titles, status, dates, and relationship joins.
- Full-text search for document bodies and transcripts.
- Embeddings for semantic similarity across captures, notes, logs, and sources.
- Hybrid ranking that combines exact match, recency, object importance, relation distance, and semantic relevance.
- Permission filter applied before ranking, even in the initial single-user architecture.

6.6 Suggested Database Tables

```
users
areas
goals
projects
project_areas
project_goals
project_logs
tasks
knowledge_notes
captures
proposal_bundles
proposed_changes
tools
```

```

sources
content_items
documents
document_blocks
relations
assets
embeddings
processing_jobs
audit_events

```

6.7 API Surface

Domain	Representative operations
Capture	create, attach, process, retry, review, dismiss.
Projects	create, update, change status, set next action, complete, archive.
Logs	create, filter, relate, convert to content seed.
Knowledge	create, merge, change mastery, relate, backlink query.
Tools	save, shortlist, start test, finish test, adopt, reject.
Content	create seed, draft, relate, publish metadata.
Search	hybrid query, filters, command resolution.
Export	single object Markdown, Area archive, full account archive.

6.8 Privacy, Security, and Trust

- Encrypt data in transit and at rest using provider capabilities.
- Use short-lived signed URLs for private R2 assets.
- Keep AI provider data-sharing settings explicit and configurable.
- Allow source deletion, transcript deletion, and complete account export.
- Preserve an audit trail of AI-created and AI-modified records.
- Do not train shared models on private user content without explicit consent.
- Support permanent deletion workflows for captures, assets, and derived embeddings.
- Design multi-tenant authorization boundaries from the start even when launching as a personal product.

6.9 Offline and Sync

The first offline guarantee should be capture durability. The mobile client stores raw captures locally, marks them queued, and uploads when connected. Later versions may support offline document editing with conflict resolution. Avoid claiming full offline-first editing in the MVP unless the conflict model is implemented and tested.

7. MVP and Delivery Plan

7.1 MVP Hypothesis

MVP hypothesis

Users will adopt the system when it reliably turns captured discoveries and active project work into organized projects, connected knowledge, and reusable content material with less effort than their existing scattered workflow.

7.2 MVP Scope

Capability	Included in MVP
Universal Inbox	Text and URL capture, raw preservation, processing state, and review queue.
Basic AI proposals	Classify project idea, tool, knowledge, reference, and content seed; suggest simple relations.
Projects	Statuses, Areas, goals, next action, overview, logs, and completion retrospective.
Knowledge	Notes, mastery, sources, project links, backlinks, and basic semantic search.
Test Later	Save, shortlist, test criteria, verdict, adopt/reject, and project link.
Content seeds	Create from log, tested tool, note, or project completion.
Weekly review	Inbox, stalled projects, missing next actions, tools, learnings, and focus selection.
Markdown export	Per-object and full-account export to R2-backed downloadable archive.

7.3 Explicitly Deferred

- Full interactive graph editing.
- Calendar replacement and advanced scheduling.
- Habit tracking and streak systems.
- Automated social publishing and analytics.
- Real-time collaboration and team permissions.
- Advanced project dependencies and resource planning.
- Fully customizable databases and arbitrary schemas.
- Dozens of block types and third-party embeds.
- Automated personal-brand analysis beyond basic recurring-theme summaries.
- Integrations with every social and read-it-later platform.

7.4 Prototype Scenario

```

Save a video about an AI tool
-> AI proposes a Tool and two Knowledge Notes
-> Accept selected proposals
-> Link the Tool to an Active Project
-> Start and finish a test
-> Record a verdict and learning
-> Add the result to the Project Log
-> Generate a Content Seed
  
```

This single scenario demonstrates the product's full differentiating loop and should be the center of the first clickable prototype and early usability testing.

7.5 Delivery Phases

Phase	Build	Exit criterion
1. Personal operating system	Areas, goals, projects, logs, captures, notes, relations.	The user relies on it daily instead of scattered notes and project files.
2. Discovery-to-knowledge	URL ingestion, video transcripts, extraction, duplicate matching, proposal review.	Saved resources reliably become useful, connected knowledge.
3. Knowledge-to-output	Tool review, retrospectives, content seeds, case-study generation.	Normal work regularly creates publishable or portfolio-ready material.
4. Identity and review	Goal dashboards, recurring themes, skill evidence, quarterly review.	The product clarifies the user's identity and career narrative.

7.6 Suggested Development Sequence

21. Implement authentication, user boundaries, PostgreSQL schema, migrations, and object CRUD.
22. Build the universal capture endpoint and durable Inbox UI.
23. Build Projects, next actions, and engineer logs.
24. Build Knowledge Notes and typed relations.
25. Build Tool lifecycle and test verdict workflow.
26. Build proposal bundles with schema-validated AI output.
27. Build hybrid search and backlinks.
28. Build content seeds and project completion retrospective.
29. Build weekly review.
30. Add Markdown export and R2 asset handling.
31. Instrument analytics and run scenario-based usability testing.

7.7 Product Analytics Events

Event	Properties
capture_created	input_type, context, platform.
capture_processed	duration, result_count, failure_stage.
proposal_reviewed	accepted_count, edited_count, dismissed_count.
project_activated	has_goal, has_target, next_action_length.
project_log_created	entry_type, related_object_count.
tool_status_changed	from_status, to_status, project_linked.
knowledge_mastery_changed	from_mastery, to_mastery, has_application.
content_seed_created	origin_type, suggested_angle, channel.
weekly_review_completed	steps_completed, focus_project_count.
project_completed	duration, output_count, note_count.

7.8 Risks and Mitigations

Risk	Impact	Mitigation
The product becomes an overbuilt Notion clone.	Scope grows without proving transformation value.	Keep fixed core entities and test the capture-to-output loop first.
AI fills the system with low-quality notes.	Trust and search quality collapse.	Use proposals, duplicate matching, confidence thresholds, and source labels.
Inbox becomes another backlog.	User avoids the system.	Fast review, bulk acceptance, processing status, and weekly cleanup.
Saved tools remain passive bookmarks.	Core behavior does not change.	Require expected use before shortlist and verdict before completion.
Graph becomes visual decoration.	High engineering cost with little utility.	Prioritize contextual connections and backlinks; defer graph editing.
Too many statuses and fields create	Capture and maintenance slow down.	Progressive disclosure and required

Risk	Impact	Mitigation
friction.		fields only at meaningful transitions.
R2-only storage limits querying.	Fragile relations and expensive indexing.	Use PostgreSQL as canonical store; export Markdown to R2.
Content generation feels generic.	Outputs do not strengthen identity.	Ground drafts in project evidence, user wording, decisions, and tested results.

7.9 Open Product Decisions

Decision	Recommended default	Validate through
Web-first or mobile-first	Web-first for full workspace; mobile capture companion in parallel.	Prototype task completion and capture frequency.
Block editor representation	Editor JSON as canonical document; Markdown export.	Round-trip fidelity tests.
Task depth	Simple tasks and milestones only in MVP.	Observe whether users need dependencies or recurring tasks.
Auto-approval	Only low-risk metadata suggestions, disabled initially.	Proposal acceptance analytics and trust interviews.
Theme personalization	One accent plus Area tokens.	Visual usability tests.
Graph availability	Read-only secondary view after contextual relations work.	Measure navigation value.

7.10 Launch Readiness Checklist

- All Active projects enforce or clearly flag a missing next action.
- Captures cannot be lost during processing failure or offline use.
- AI proposals are editable and reversible before approval.
- Duplicate detection exists for notes, tools, and sources.
- Every AI-generated section is labeled.
- Project completion produces a retrospective and output opportunity.
- Exports include stable IDs, relations, and assets.
- Search respects user authorization and deletion state.
- Empty, loading, failure, and partial-result states are designed and tested.
- The primary prototype scenario can be completed without explanation from the designer.

Appendix A. Core User Stories

Domain	User story
Capture	As a user, I can save an idea or URL immediately so I do not interrupt what I am doing.
Review	As a user, I can see exactly what AI wants to create and approve only the useful parts.
Project	As a user, I can see the next action for every active project so I know how to move forward.
Log	As a developer, I can document decisions and experiments so my reasoning is not lost.
Knowledge	As a student, I can explain a concept in my own words and connect it to where I applied it.
Tools	As a curious builder, I can track why I saved a tool, test it, and record a verdict.
Content	As a creator, I can turn real work into content without starting from a blank page.
Identity	As a user, I can see recurring themes and evidence across my projects so my personal brand becomes clearer.
Review	As a user, I can complete a guided weekly review so the system stays current.
Portability	As a user, I can export readable Markdown and assets so I retain ownership of my information.

Appendix B. Example Templates

Project Brief

Outcome
What exists when this project is complete?

Why it matters
Which Goals and Life Areas does this support?

Success criteria
What observable conditions define completion?

Constraints
Time, tools, scope, or dependencies.

Next action
The next physical, visible step.

Experiment Log

Hypothesis

Method

Result

Interpretation
What changed in my understanding?

Next test

Knowledge Note

Plain-language explanation

How it works

Example

Common mistakes

My explanation

Where I applied it

Prerequisites and related concepts

Sources

Project Retrospective

What I made

What worked

What did not

What I learned

What I would change

What this demonstrates

What can become content or portfolio evidence

Appendix C. Traceability to Original Concept

Original intent	Specification response
Create cohesion across personal brand, content, development, school, and life.	Life Areas, multi-area projects, goals, Identity Studio, and evidence-linked outputs.
Track projects doing, wanting to do, and completed.	Explicit Project lifecycle with Idea, Planned, Active, Paused, Completed, and Archived.
Give every project an engineer's log.	Typed log-entry records, templates, chronology, search, and content conversion.
Build an Obsidian-like knowledge graph for hard knowledge.	Typed Knowledge Notes, mastery, backlinks, semantic relations, and secondary graph view.
Catalog cool tools to test later.	Purposeful Tool lifecycle with expected use, test criteria, verdict, and adoption state.
Generate content from tested tools and projects.	Content seeds and provenance-linked content pipeline.
Create projects from natural-language ideas.	Universal capture with AI classification and project proposal.
Digest videos into concepts and knowledge.	Source ingestion, transcript processing, concept extraction, duplicate matching, and approval.
Use /braindump to classify information.	Immutable captures, proposal bundles, and granular user review.
Store Markdown in R2 and render like Notion.	PostgreSQL canonical model plus block editor, Markdown export, R2 assets and snapshots.

Original intent	Specification response
Everything is a movable block.	Generic blocks plus product-specific Decision, Experiment, Learning, and Content Seed blocks.

Appendix D. Source

Primary source: "Life Organization System," a three-page concept brief supplied by the product owner. The brief defines the desired identities, project list, engineer log, knowledge graph, test-later catalog, AI capture commands, Markdown/R2 direction, and Notion-style block model. This specification expands those ideas into product, design, data, and implementation requirements.

End of specification.